ROSE 5710 — Assignment 2

(Deadline Dec 2nd 23:59:59)

Environment Setup

1. Use the same VirtualBox environment from Assignment 1.
2. Template file: Copy the file from Blackboard to ~/ros2_ws/src. You will have two new folders hw2 and hw2_msg. The structure should be the same as

```
rose5710@ROSE5710:~/ros2_ws/src$ ls
hw1  hw2  hw2_msg
```

Only numpy, rclpy and built-in Python packages are allowed.

ROS2 Reference: https://docs.ros.org/en/humble/index.html

The ROS2 working space is already created, you need to copy the template codes from blackboard to the ROS2 working space. Your task is to finish the following problems in the node template we provided.

You should declare the usage and explain why and how you used it in your code using comment if you use any AI to help you to finish the code include but not limited to online chat box and AI powered editor/plugin.

Problem 1: Inverse Kinematics (IK)

- File: ~/ros2_ws/src/hw2/hw2/SOARM101.py, ~/ros2_ws/src/hw2/hw2/publisher.py, ~/ros2_ws/src/hw2/hw2/subscriber.py
- Task: Implement an Inverse Kinematics solver that converts an end-effector pose (position + orientation) into the robot's joint angles.
    1. Subscribe to the tf2 messages from hw1.
        - **Topic Name**: /hw2_q1_tf2_<your_SID>

- **Message Type**: TF2, (TransformStamped): https://docs.ros.org/en/foxy/Tutorials/Intermediate/Tf2/Tf2-Main.html

2. **Compute IK**. You must write code for both Analytical IK and Numerical IK. You should use Jacobian inverse for Numerical IK.
   - Input
     - Desired end-effector position: $[x, y, z]$ in mm.
     - Desired end-effector quaternion.
   - Output
     - Joint angles as a list or NumPy array.
3. **Publish Result Joint Angles**:
   - **Topic Name**: /hw2_q1_ana_<your_SID>, /hw2_q1_num_<your_SID>
   - **Message Type**: Float64MultiArray.

Problem 2: Cubic Trajectory Generation

- File ~/ros2_ws/src/hw2/hw2/cubic_trajectory_generation.py, ~/ros2_ws/src/hw2_msg/msg/TrajectoryCommand.msg
- Task: Implement a self-defined ROS2 message. Receive the message and generate a cubic trajectory for the robot's end-effector in **task space**, then convert it into a **joint space trajectory** using your IK solver. Publish the generated trajectory in given frequency.
- **Units:** Positions in **mm**; angles in **radians**; time in **seconds**; frequency in **Hz**.

1. Create a self-defined message with the following elements
   - The following variables should in type of float64[3]: start_position, start_euler_xyz, end_position, end_euler_xyz, v_start_pos, v_end_pos, w_start_euler, w_start_euler
   - The following variables should be in type of float64: duration, frequency.
2. Declare your message in CMakeLists.txt and package.xml.
3. Subscribe to the messages that contain the trajectory requirements.
   - **Topic Name**: /hw2_q2_<your_SID>

        - **Message Type**: hw2_msg/TrajectoryCommand

    4. **Task Space Trajectory Generation**: Generate the task space trajectory based on the command received.

    5. **Joint Space Trajectory Conversion**: Convert your task space trajectory into joint space trajectory by using the ik_solver from q1.

    6. **Publish Joint Space Trajectory**:

        - **Topic Name**: /hw2_q2_joint_cmd_<your_SID>

        - **Message Type**: Float64MultiArray

        - At each tick t_i, publish the **current 6×1 joint vector** at given frequency. Do **not** publish the entire flattened trajectory at once.

    7. **Save the trajectory to file:** Use the save_trajectory_to_file function in save_trajectory_to_file.py

(Hint**:** Use shortest path unwrap for Euler angles across ±π to avoid jumps. SLERP/quaternions are *not* required.)

Report

In addition to the code submission, each student must submit a short report (PDF) titled hw2_report_<your_SID>.pdf together with the source code.
The report should be concise (about 1–2 pages) and include the following sections:

1. **Difference between Analytical IK and Numerical IK**

    - Explain the **conceptual and practical differences** between the two approaches.

    - Discuss aspects such as formulation, computational efficiency, convergence behavior, and suitability for different robot types.

2. **Implementation Description**

    - Summarize **what you have implemented** in this assignment:

        - How your **IK solver** is designed or structured.

            1. For numerical methods, describe your stopping criteria, step size, and error tolerance.

            2. For analytical methods, describe the key geometric reasoning or transformation chain.

- How you generated the **cubic task-space trajectory** and converted it to **joint-space trajectory**.

Submission

- The code should only subscribe and publish within ROS. Do not include any terminal input handling.
- Ensure your code runs without any syntax errors. We will not provide corrections.
- Do not copy others' work. Submissions will be checked for plagiarism.
- Copy the message file from hw2_msg/msg to hw2/msg
- Compress only the folder: ~/ros2_ws/src/hw2/hw2/. The structure of the zip file should be the same with:

```
hw2
├── hw2
│   ├── cubic_trajectory_generation.py
│   ├── __init__.py
│   ├── publisher.py
│   ├── save_trajectory_to_file.py
│   ├── SOARM101.py
│   └── subscriber.py
├── hw2_report_1155xxxxxx.pdf
├── LICENSE
├── msg
│   └── TrajectoryCommand.msg
├── package.xml
├── resource
│   └── hw2
├── setup.cfg
├── setup.py
└── test
```

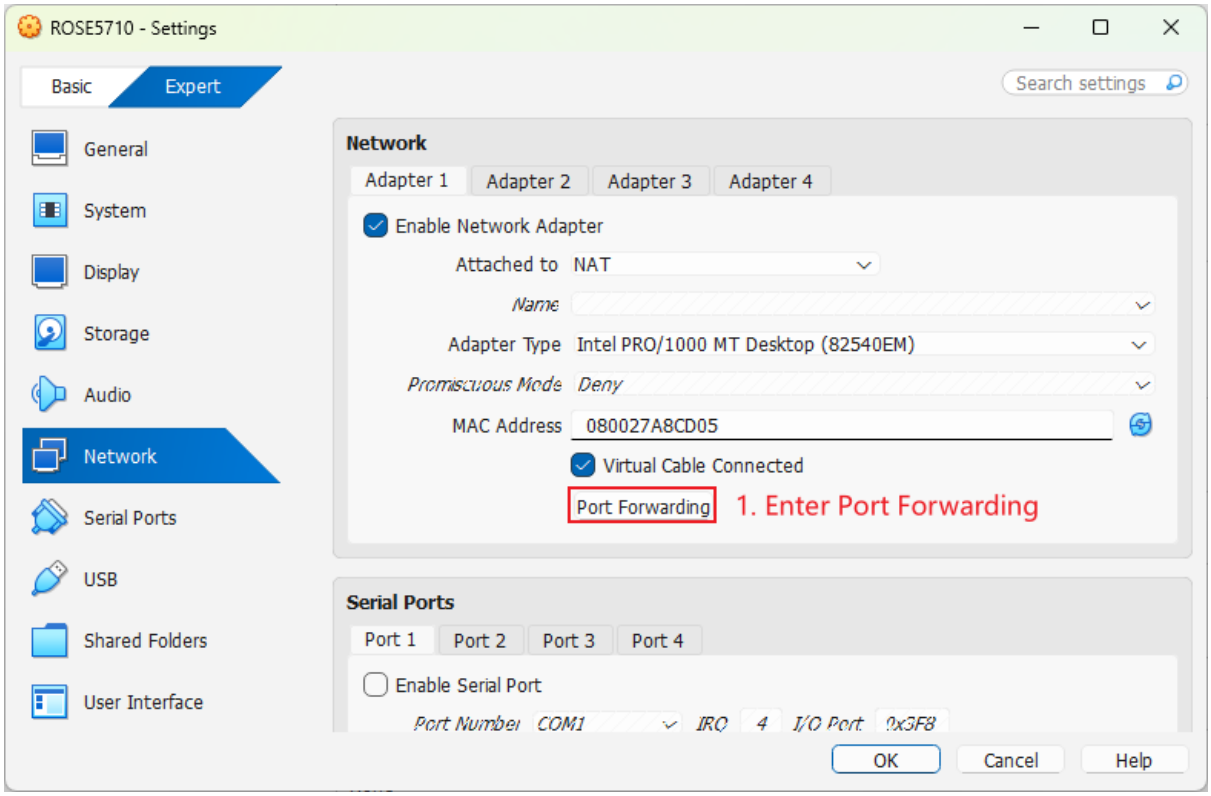
- Rename the zip file to: hw2_<your_SID>.zip (i.e. hw2_1155234567.zip)

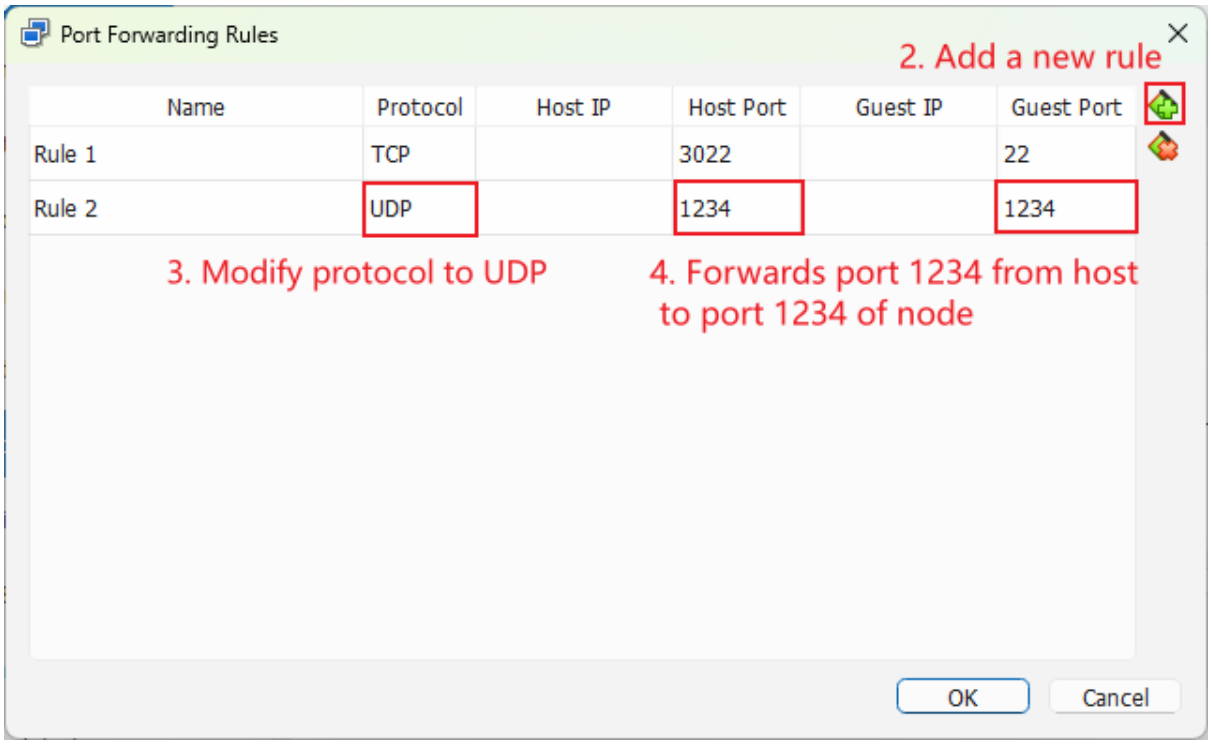
Validation

- Task: Please follow the steps, use the robot arm simulator (You can find it on Blackboard) to finish the validation:

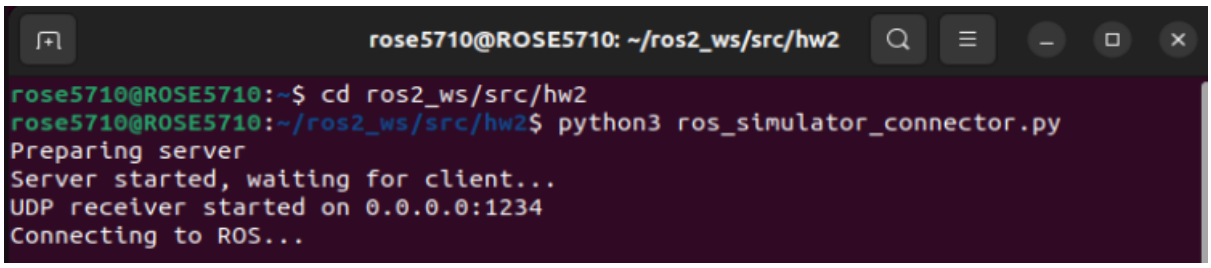
1. **Environment Set Up.**

1) Download the ros_simulator_connector.py to your virtual box.
2) Download the robot arm simulator to your own system from BlackBoard. Uncompress it and ensure the path contains ASCII characters only.
3) Add a port forwarding rule in VirtualBox Manager, which forwards port **1234** from host to port **1234** of node, with protocol **UDP**.

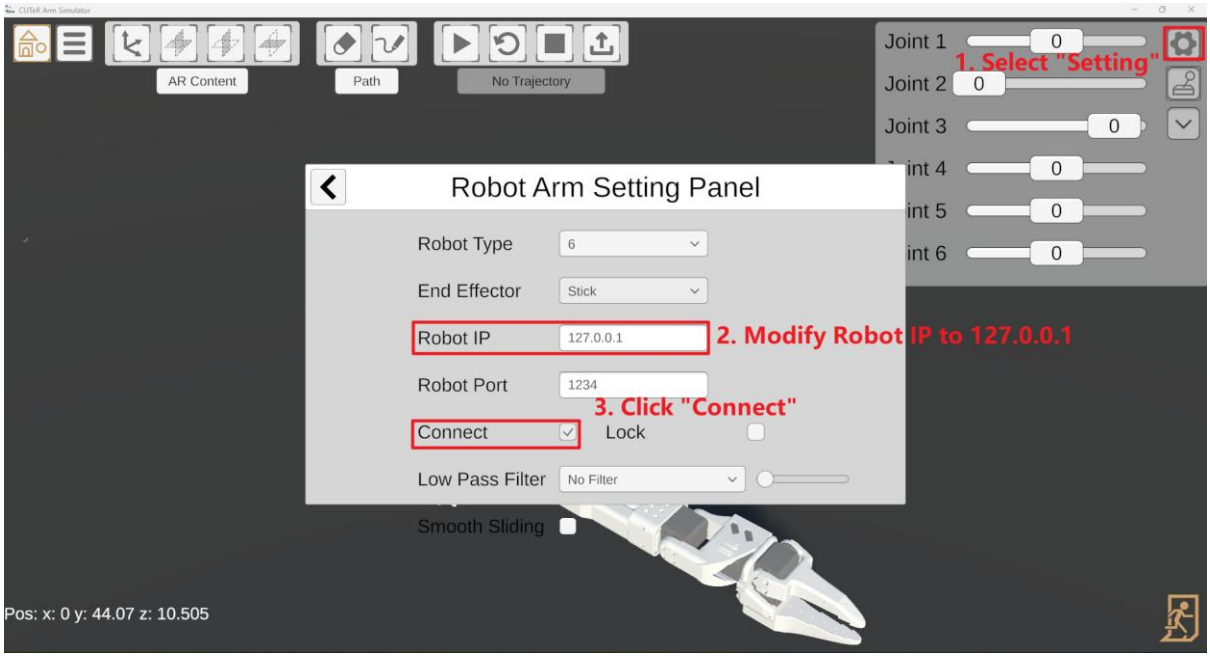




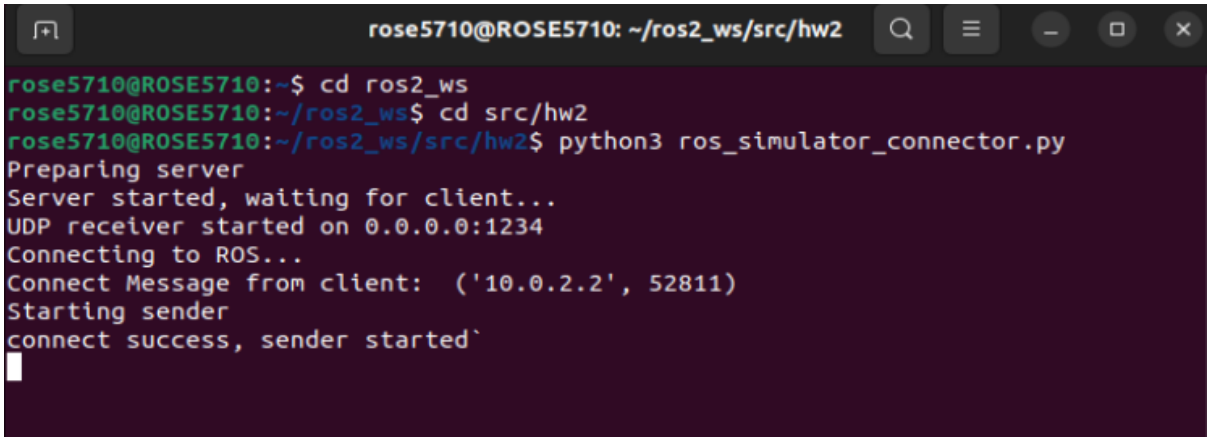
2. **Run the Connector and Connect the Simulator**
   - Go the path where you store the connector, execute the script using: *python3 ros_simulator_connector.py* in terminal



   - Run **CUTeR Arm Simulator**.
   - Click the "**Setting**" button as figure below shown, modify the Robot IP to **127.0.0.1**, click "**connect**"



   - Once you connect the simulator, you will see:



3. Change the topic name from /hw2_q2_joint_cmd_<your_SID> to /joint_angles_command in task 2.
4. Execute your trajectory generation node.
5. Publish a TrajectoryCommand message to your node.

If your code is correct, the robot arm simulator should move smoothly as your trajectory.